Bloco 5 - Metodologia de pentest

Ciclo Profissional de Pentest



Depois da correcao vem o reteste.

Aula 31 - Metodologia de Pentest

Pentest profissional comeca por autorizacao e escopo, segue por reconhecimento, hipoteses, validacao minima, avaliacao de impacto, relatorio e reteste. Ferramenta e apenas parte do processo.

Pontos-chave

- Defina o que esta dentro e fora do escopo.
- Pare quando a evidencia minima for suficiente.
- Documente horarios, endpoints e resultados.

Exemplo tecnico

Codigo / comando

```
function noEscopo(url, hosts) {
  return hosts.includes(new URL(url).hostname);
}
```

Portugol - equivalente didatico

```
funcao logico noEscopo(cadeia url, lista hosts)
    retorne host(url) pertence hosts
fimfuncao
```

Aula 32 - Reconhecimento e Superficie de Ataque

Superficie de ataque e o conjunto de sistemas, rotas, APIs, parametros, portas e tecnologias expostos e autorizados para avaliacao. Reconhecimento organiza esse inventario antes de testes mais profundos.

Pontos-chave

- Mapeie hosts e aplicacoes.
- Catalogue rotas e parametros.
- Diferencie ativos proprios de terceiros.

Exemplo tecnico

Codigo / comando

```
const u = new URL("https://app.exemplo.com/produto?id=10");
console.log(u.hostname, u.pathname, [...u.searchParams.keys()]);
```

Portugol - equivalente didatico

```
host <- obterHost(url)
rota <- obterRota(url)
parametros <- obterParametros(url)
mostrar(host, rota, parametros)
```

Aula 33 - De Pista a Hipotese

Um scanner deve diferenciar observacao de vulnerabilidade. Um parametro id e uma pista; uma versao aparente pode sugerir um CVE; nenhuma das duas confirma impacto sozinha.

Pontos-chave

- INFO: observacao.
- POSSIVEL: merece validacao.
- CONFIRMADO: exige evidencia suficiente.

Exemplo tecnico

Codigo / comando

```
function classificar(evidencia) {
  if (evidencia.confirmada) return "CONFIRMADO";
  if (evidencia.indicio) return "POSSIVEL";
  return "INFO";
}
```

Portugol - equivalente didatico

```
se evidencia.confirmada entao
    retorne "CONFIRMADO"
senao se evidencia.indicio entao
    retorne "POSSIVEL"
senao
    retorne "INFO"
fimse
```

Aula 34 - Validacao de Impacto

Severidade depende de contexto: dados afetados, autenticacao necessaria, alcance, capacidade de leitura/alteracao e facilidade. O objetivo e provar risco sem causar dano desnecessario.

Pontos-chave

- Compare impacto e probabilidade.
- Dados de terceiros elevam impacto.
- Nao destrua ou extraia mais do que o necessario.

Exemplo tecnico

Codigo / comando

```
function score(impacto, probabilidade) {
  return Math.min(10, impacto * probabilidade);
}
```

Portugol - equivalente didatico

```
pontuacao <- impacto * probabilidade
se pontuacao > 10 entao pontuacao <- 10 fimse
```

Aula 35 - Relatorio Profissional e Reteste

Um finding profissional inclui titulo, severidade, endpoint, descricao, pre-requisitos, reproducao, resultado observado, esperado, impacto, evidencia e recomendacao. Depois da correcao, o reteste determina se foi resolvido.

Pontos-chave

- Evite linguagem sensacionalista.
- Remova dados sensiveis desnecessarios da evidencia.
- Registre status RESOLVIDO ou AINDA PRESENTE no reteste.

Exemplo tecnico

Codigo / comando

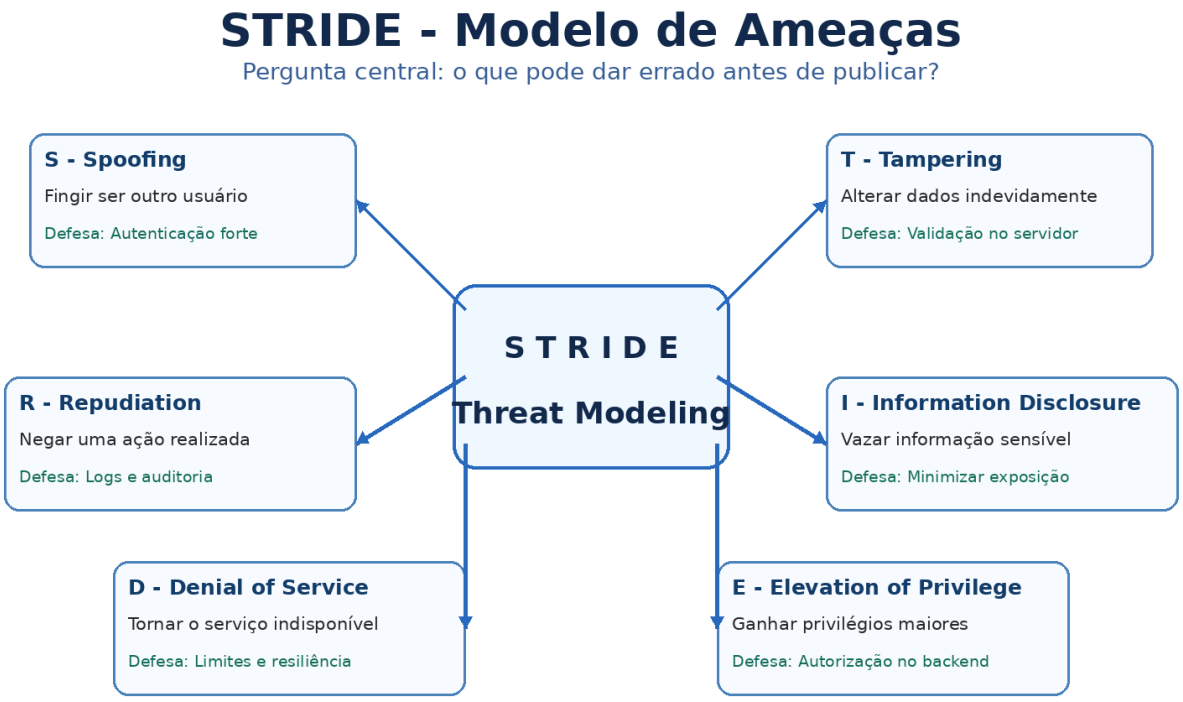
```
const finding = {
  title: "Controle de acesso inadequado",
  severity: "HIGH",
  endpoint: "/profile",
  recommendation: "Validar autorizacao no servidor"
};
```

Portugol - equivalente didatico

```
registro finding
    cadeia titulo
    cadeia severidade
    cadeia endpoint
    cadeia recomendacao
fimregistro
```

Um bom relatorio separa observacao, hipotese, evidencia, impacto e recomendacao.

Bloco 6 - Desenvolvimento seguro e projeto final



Bloco 6 - Desenvolvimento seguro e projeto final

Aula 36 - Threat Modeling com STRIDE

Threat Modeling pergunta o que pode dar errado antes da publicacao. STRIDE cobre Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service e Elevation of Privilege.

Pontos-chave

- Modele fluxos e fronteiras de confianca.
- Relacione cada ameaca a controles.
- Revise quando arquitetura mudar.

Exemplo tecnico

Codigo / comando

```
const stride = ["Spoofing","Tampering","Repudiation","Information Disclosure","Denial of
Service","Elevation of Privilege"];
for (const threat of stride) console.log(threat);
```

Portugol - equivalente didatico

```
ameacas <- ["Spoofing", "Tampering", "Repudiation", "Information Disclosure", "Denial of Service",
"Elevation of Privilege"]
para cada ameaca em ameacas faca escreva(ameaca) fimpara
```

Aula 37 - Secure Coding

Secure Coding incorpora validacao, autenticacao, autorizacao, queries parametrizadas, protecao de segredos, logs adequados e regras de negocio no desenvolvimento diario.

Pontos-chave

- Cliente pede; servidor valida e decide.
- Segredos ficam fora do codigo.
- Falhas internas sao logadas sem vazar detalhes ao cliente.

Exemplo tecnico

Codigo / comando

```
const ProductSchema = z.object({ productId: z.number().int().positive(), quantity:
z.number().int().min(1).max(100) });
const data = ProductSchema.parse(req.body);
```

Portugol - equivalente didatico

```
se produtoId <= 0 ou quantidade < 1 ou quantidade > 100 entao
    responder(400)
senao
    processar(produtoId, quantidade)
fimse
```

Aula 38 - SAST, DAST, SCA e Secret Scanning

SAST analisa codigo; DAST analisa aplicacao rodando; SCA analisa dependencias; Secret Scanning procura chaves e tokens. As camadas se complementam.

Pontos-chave

- npm audit e um exemplo de SCA no ecossistema Node.
- Scanner proprio esta mais proximo de DAST/recon.
- Segredo exposto deve ser revogado, nao apenas apagado.

Exemplo tecnico

Codigo / comando

```
const checks = ["SAST", "SCA", "Secret Scanning", "DAST"];
for (const check of checks) executar(check);
```

Portugol - equivalente didatico

```
verificacoes <- ["SAST", "SCA", "Secret Scanning", "DAST"]
para cada item em verificacoes faca executar(item) fimpara
```

Aula 39 - DevSecOps

DevSecOps coloca verificacoes de seguranca no ciclo de entrega: codigo, testes, SAST, SCA, segredo, build, deploy, DAST e monitoramento. Falhas criticas podem bloquear publicacao.

Pontos-chave

- Detectar cedo reduz custo.
- Pipeline deve ser reproduzivel.
- Monitoramento continua depois do deploy.

Exemplo tecnico

Codigo / comando

```
async function pipeline() {
  await testes(); await sast(); await sca();
  await build(); await deploy(); await dast();
}
```

Portugol - equivalente didatico

```
procedimento pipeline()
    testes()
    sast()
    sca()
    build()
    deploy()
    dast()
fimprocedimento
```

Aula 40 - Projeto Final - Pentest Completo de Laboratorio

O fechamento do primeiro ciclo e um pentest de laboratorio propositalmente vulneravel. O trabalho combina escopo, reconhecimento, scanner, testes manuais, evidencia, correcao e reteste.

Pontos-chave

- Priorize raciocinio antes de payloads.
- Produza relatorio inicial e final.
- O objetivo e demonstrar encontrar e corrigir problemas.

Exemplo tecnico

Codigo / comando

```
const report = {
  target: "localhost",
  findings: [],
  retest: []
};
fs.writeFileSync("report.json", JSON.stringify(report, null, 2));
```

Portugol - equivalente didatico

```
relatorio.alvo <- "localhost"
relatorio.achados <- listaVazia()
relatorio.reteste <- listaVazia()
salvarJSON(relatorio)
```

Entrega final sugerida: relatorio inicial + correcoes + reteste + relatorio final.